



Lab Sheet 3

Data Structures– I Lab(BTCS209P)

B. Tech.(AIML,CyberSecurity,andDataScience)

III Semester (Session 2026–2027)

Objective

To understand the structure of a singly linked list and implement its fundamental operations using C++.

Prerequisites

- Basic knowledge of C++ programming, functions, loops, and conditional statements.
- Understanding of pointers and dynamic memory allocation.
- Basic knowledge of arrays and data structures.
- A C++ compiler (Code::Blocks / Visual Studio Code / Dev-C++).

Task

Open your C++ IDE and perform the following tasks.

Task 1: Create and Traverse a Singly Linked List

Write a C++ program to create a singly linked list using user-entered data, display all nodes by traversing the list and count the total number of nodes.

Task 2: Insert a Node

Write a C++ program to insert a new node at the beginning, at the end, and at a given position in a singly linked list.

Task 3: Delete a Node

Write a C++ program to delete a node from the beginning, from the end, and from a given position in a singly linked list.

Task 4: Search and Count

Write a C++ program to search for a given element in a singly linked list.

Task 5: Conceptual Questions

1. What is a singly linked list? How is it different from an array?
2. What are the two main parts of a node in a singly linked list?
3. Why is a pointer required in a singly linked list?
4. What does the next pointer of the last node contain?
5. What is the difference between head and a normal node pointer?
6. Why is insertion at the beginning of a singly linked list considered efficient?
7. What happens if the head pointer is NULL?
8. Why is traversal required before searching for an element in a singly linked list?
9. What is the time complexity of traversal, searching, insertion at beginning, and deletion at beginning?
10. What problems can occur if dynamically allocated nodes are not properly deleted?



Instructions

- Write each programming task as a separate C++ program unless a menu-driven program is specifically requested.
- Use a node structure/class containing data and a next pointer.
- Use user-entered data to create the linked list.
- Handle empty-list and invalid-position cases appropriately.
- Use meaningful variable names, proper indentation, and comments.
- Compile and execute each program successfully.
- Save each program with an appropriate filename.

Submission

- Take screenshots of source code and output for each programming task.
- Include answers to the conceptual questions in the practical record.
- Save all C++ source files in a single folder.
- Submit the folder along with the screenshots to the course instructor.

LAB SHEET --3

Task

Open your C++ IDE and perform the following tasks.

Task 1: Create and Traverse a Singly Linked List

Write a C++ program to create a singly linked list using user-entered data, display all nodes by traversing the list and count the total number of nodes.

Solution

```
#include <iostream>
```

```
using namespace std;
```

```
struct Node{  
    int data;  
  
    Node * next;
```

```
};
```

```
int main()
```

```
{ Node*head=NULL;
```

```
    Node *temp;
```

```
    Node *newNode;
```

```
    int n,value,count = 0;
```

```
    cout<<"Enter number of nodes: ";
```

```
    cin >> n;
```

```
    for(int i=1;i<= n; i++) {
```

```
        newNode=new Node;
```

```
        cout<<"Enter data for node " << i << ": ";
```

```
        cin >> value;
```

JAYSHREE SAINI (CYBER SECURITY) ,(CU25230014),E

```

newNode->data = value;
newNode->next = NULL;
if (head == NULL){
    head = newNode;
}
else{
    temp = head;
    while (temp->next != NULL){
        temp = temp->next;
    }
    temp->next = newNode;
}
}
cout << "\nLinked List: ";
temp = head;
while (temp != NULL){
    cout << temp->data << " -> ";
    count++;
    temp = temp->next;
}
cout << "NULL\n";
cout << "Total number of nodes = " << count << "\n";
return 0;
}

```

Task 2: Insert a Node

Write a C++ program to insert a new node at the beginning, at the end, and at a given position in a singly linked list.

Solution

```
#include <iostream>
using namespace std;
struct Node
{
    int data;
    Node *next;
};
int main(){
    Node *head = NULL;
    Node *temp;
    Node *newNode;
    int n, value, choice, position;
    cout << "Enter number of nodes: ";
    cin >> n;
    for (int i = 1; i <= n; i++){
        newNode = new Node;
        cout << "Enter data: ";
        cin >> value;
        newNode->data = value;
        newNode->next = NULL;
        if (head == NULL) {
            head = newNode;
        }
        else {
            temp = head;
```

```

while (temp->next != NULL)
{temp = temp->next;
}
temp->next = newNode;
}
}

cout << "\n1. Insert at Beginning\n";
cout << "2. Insert at End\n";
cout << "3. Insert at Given Position\n";
cout << "Enter your choice: ";
cin >> choice;

cout << "Enter value to insert: ";
cin >> value;

newNode = new Node;
newNode->data = value; newNode->next = NULL;

if (choice == 1){
newNode->next = head;
head = newNode;
cout << "Node inserted at beginning.\n";
}

```

```

else if (choice == 2){
if (head == NULL){
head = newNode;
}
else{
temp = head;
while (temp->next != NULL) {
temp = temp->next;
}
temp->next = newNode;
}
cout << "Node inserted at end.\n";
}
else if (choice == 3){
cout << "Enter position: ";
cin >> position;
if (position == 1) {
newNode->next = head;
head = newNode;
}
else{
temp = head;

```

```

for (int i = 1; i < position - 1 && temp != NULL; i++)
{temp = temp->next;}
if (temp == NULL){
cout << "Invalid position.\n";
delete newNode;}
else{
newNode->next = temp->next;
temp->next = newNode;
cout << "Node inserted at given position.\n";
}
}
}
else{
cout << "Invalid choice.\n";
}
cout << "\nLinked List: ";
temp = head;
while (temp != NULL){
cout << temp->data << " -> ";
temp = temp->next;
}
cout << "NULL\n";
return 0;
}

```

Task 3: Delete a Node

Write a C++ program to delete a node from the beginning, from the end, and from a given position in a singly linked list.

Solution

```
#include <iostream>

using namespace std;

struct Node{
    int data;
    Node *next;
};

int main(){
    Node *head = NULL;
    Node *temp;
    Node *newNode;
    Node *deleteNode;
    int n, value, choice, position;
    cout << "Enter number of nodes: ";
    cin >> n;
    for (int i = 1; i <= n; i++) {
        newNode = new Node;
        cout << "Enter data: ";
        cin >> value;
        newNode->data = value;
        newNode->next = NULL;
```

```

if (head == NULL) {
    head = newNode;
}
else{
    temp = head;
    while (temp->next != NULL) {
        temp = temp->next;
    }
    temp->next = newNode;
}
}

cout << "\n1. Delete from Beginning\n";
cout << "2. Delete from End\n";
cout << "3. Delete from Given Position\n";

cout << "Enter your choice: ";
cin >> choice;
if (head == NULL) {
    cout << "List is empty.\n";
}
else if (choice == 1) {
    deleteNode = head;
    head = head->next;
    delete deleteNode;
    cout << "Node deleted from beginning.\n";
}

```

```

elseif(choice == 2){
    if(head->next == NULL) {
        delete head;
        head = NULL;
    }
    else {
        temp = head;
        while (temp->next->next != NULL) {
            temp = temp->next;
        }
        delete temp->next;
        temp->next = NULL;
    }
    cout<< "Node deleted from end.\n";
}

elseif(choice == 3){
    cout<< "Enter position: ";
    cin>> position;
    if(position == 1) {
        deleteNode = head;
        head = head->next;
        delete deleteNode;
        cout << "Node deleted from position 1.\n";
    }

```

```

else{
    temp=head;
    for(int i=1;i< position - 1 && temp != NULL; i++)    {
        temp = temp->next;
    }
    if(temp==NULL || temp->next == NULL) {
        cout<<"Invalid position.\n";
    }
    else {
        deleteNode= temp->next;
        temp->next= deleteNode->next;
        delete deleteNode;
        cout<<"Node deleted from given position.\n";
    }
}
}
else {
    cout << "Invalid choice.\n";
}
cout<<"\nLinkedList: ";
temp = head;
while(temp!=NULL) {
    cout<<temp->data << " -> ";
}

```

```
temp = temp->next;  
}  
cout << "NULL\n";  
return 0;  
}
```

Task 4: Search and Count

Write a C++ program to search for a given element in a singly linked list.

Solution

```
#include <iostream>

using namespace std;

struct Node{
    int data;
    Node *next;
};

int main(){
    Node *head = NULL;
    Node *temp;
    Node *newNode;
    int n, value, search;
    int count = 0;
    int position = 1;
    cout << "Enter number of nodes: ";
    cin >> n;
    for (int i = 1; i <= n; i++) {
        newNode = new Node;
        cout << "Enter data for node " << i << ": ";
        cin >> value;
        newNode->data = value;
        newNode->next = NULL;
        JAYSHREE SAINI (CYBER SECURITY) ,( CU25230014),E
```

```

if(head==NULL) {
    head=newNode;
}
else {
    temp = head;
    while(temp->next != NULL) {
        temp= temp->next;
    }
    temp->next = newNode;
}
}
cout<<"\nEnter element to search: ";
cin >> search;
temp = head;
while(temp!= NULL) {
    if(temp->data == search) {
        cout<<"Element found at position "
            <<position << "\n";
        count++;
    }
    temp = temp->next;
    position++;
}

```

```
if(count == 0)
{
    cout << "Element not found.\n";
}
else
{
    cout << "Element occurs " << count
        << " time(s).\n";
}

return 0;
}
```